

Pagina 1

1. Identificarea actorilor și descrierea unui caz de utilizare

Actori:

- Administrator
- Profesor
- Student

Caz de utilizare exemplu: „Profesorul corectează tema unui student”

Descriere caz utilizare:

- Actor principal: Profesor
- Scop: Corectarea temei trimise de student
- Precondiții: Studentul este înscris la curs și a trimis tema
- Pași:
 1. Profesorul se autentifică în sistem
 2. Accesează cursul
 3. Selectează tema trimisă de student
 4. O evaluează și introduce feedback și nota
 5. Sistemul notifică studentul cu rezultatul evaluării

2. Arhitectura și diagrama de clase

Arhitectura propusă:

- Arhitectură client-server
- Pattern de acces la bază de date: Table Data Gateway
- Notificare în timp real: WebSockets sau pattern Observer
- Design pattern general: MVC (Model-View-Controller), dar puteti sa il extindeti usor la Front End – Backend layered architecture, microservicii etc...

Diagrama de clase (descrișă textual):

- User
 - id
 - nume
 - email

- parola
 - rol (student/profesor/admin)
- Curs
 - id
 - titlu
 - descriere
 - profesor_id
- Tema
 - id
 - titlu
 - descriere
 - curs_id
 - termen_limita
- TemaStudent
 - id
 - tema_id
 - student_id
 - continut
 - nota
 - feedback
- Notificare
 - id
 - utilizator_id
 - mesaj
 - data

3. Diagrama bazei de date

Tabele principale:

- users (id, nume, email, parola, rol)
- cursuri (id, titlu, descriere, profesor_id)

- teme (id, titlu, descriere, curs_id, termen_limita)
 - teme_studenti (id, tema_id, student_id, continut, nota, feedback)
 - notificari (id, utilizator_id, mesaj, data)
-

Pagina 2

1. Acelasi still architecture, FE-BE, microservicii etc...
2. State Pattern

Context: Motorul

Fiecare stare este o clasă care implementează o interfață MotorState

Contextul delegă acțiunea către starea curentă

Singleton pentru reutilizare:

- Fiecare stare poate fi implementată ca singleton pentru a evita mai multe instanțe.

interface MotorState {

void handle(Motor context);}

class IdleState implements MotorState {

public void handle(Motor context) {
context.setState(new StopState()); } }

class StopState implements MotorState {

public void handle(Motor context) {
context.setState(new StartState()); } }

class StartState implements MotorState {

public void handle(Motor context) {
context.setState(new ChangeSpeedState()); } }

class ChangeSpeedState implements MotorState {

public void handle(Motor context) {
context.setState(new StopState()); } }

class Motor {

private MotorState state;

public Motor() {

```
state = new IdleState(); // Inițial }  
public void setState(MotorState state) {  
    this.state = state; }  
public void request() {  
    state.handle(this); }}
```

Pagina 3

1. Sistem de închiriere auto – Arhitectură stratificată (3 layere)

Layerele clasice:

1. Presentation Layer (UI Web): utilizatorul poate:
 - căuta după tip de mașină, locație, dată
 - vedea detalii despre mașină
 - face / anula o rezervare
2. Business Logic Layer:
 - validează perioada, disponibilitatea, etc.
 - gestionează regulile de închiriere
 - interacționează cu data layer
3. Data Access Layer:
 - interacționează cu baza de date pentru operațiile CRUD

Avantaje:

- Separare clară a responsabilităților
- Scalabilitate mai ușoară (ex: interfață mobilă nouă = doar Presentation Layer diferit)

Dezavantaje:

- Mai mult cod de legătură
- Timp de dezvoltare inițial mai lung

2. Design pentru Business Logic Layer – 2 variante

Variantă 1: Service Classes + DAO pattern

- Fiecare entitate are un DAO pentru acces DB și un Service pentru logica aplicației.

Pro:

- Ușor de testat
- Clar separare logică

Con:

- Multe clase mici de gestionat

VARIANTĂ 2: Domain Model + Repository pattern

- Encapsulează logica în obiectele domeniului (ex: CarReservation)
- Use-case-urile acționează pe aceste entități

Pro:

- Obiectele de domeniu devin inteligente
- Extensibilitate mare

Con:

- Mai complex pentru aplicații simple

3. Componentă complexă ce salvează rezultate criptate într-un fișier

a) Ce design pattern asigură transparența criptării?

Decorator Pattern

- Împachetează clasa ResultWriter într-un EncryptedWriter care adaugă criptare.

Pro: Componentele de bază rămân neschimbate, transparență pentru client.

b) Ce pattern permite înlocuirea ușoară a algoritmului de criptare?

Strategy Pattern

- Definește o interfață EncryptionStrategy și mai multe implementări: AESEncryption, RSAEncryption, etc.

4. Modificare a unei metode statice de sortare pentru long arrays

a) Extindere pentru long[]

fără metoda statică

```
public static void quickSort(int[] arr) {...}
```

Refactorizezi folosind Generics + Strategy:

```
public interface Sorter<T> {
    void sort(T[] array);
}
```

```
public class QuickSort implements Sorter<Integer> {  
    public void sort(Integer[] array) {...}  
}
```

Apoi pentru long[] folosești un LongSorter, poate convertit la Long[].

b) Se va folosi

Design pattern: Strategy

Generic programming: pentru a face sortarea tip-independentă

Nu folosim inheritance, deoarece sortarea este comportament, nu „este un”.

Pagina 4

Utilizatorul poate:

- Vizualiza produse
 - Adăuga/șterge/modifica cantități în ShoppingCart
 - La checkout, coșul devine Order (care e final)
1. Diagrama bazei de date
 2. Layered architecture, diagrama aplicatiei, si diagrama layere backend, db
 3. Deployment diagram
 4. Package -> layerele
 5. Secventa
-

Pagina 5

1. Încalcă Open/Closed Principle (OCP) și Liskov Substitution Principle (LSP). Codul e dependent de tipurile concrete și se modifică pentru fiecare subtip nou de Car. Folosește polimorfism cu o metodă abstractă în Car
2. Vezi diferite design pattern
3. Bridge Pattern – separă abstracția (ex. logică de afișare viteză) de implementare (ex. tipul de ecran)

```
interface Display {
```

```
    void showSpeed(int speed);}
```

```
class LEDDisplay implements Display { ... }
```

```
class BigScreenDisplay implements Display { ... }
```

```
abstract class Speedometer {
```

```
    protected Display display;
```

```

public Speedometer(Display d) { this.display = d; }

abstract void updateSpeed(int speed);

class BoatSpeedometer extends Speedometer {

    void updateSpeed(int speed) {

        display.showSpeed(speed);  }}

```

4. Composite Pattern

Tratează File și Directory uniform printr-o interfață comună (FileSystemComponent)

Directory conține o listă de componente

Pagina 6

Ex1.

Actori:

- Client
- Restaurant admin
- Sistem de plată (extern)

Use Cases pentru Client:

- Vizualizează restaurante
- Deschide comandă (individuală sau de grup)
- Adaugă item la comandă
- Participă la comandă
- Checkout comandă
- Vizualizează status comandă

Use Cases pentru Admin restaurant:

- CRUD iteme meniu
- Vizualizare rapoarte
- Marca livrare
- Vizualizează comenzi active

Ex2. Arhitectura FE, BE, DB, layered

Composite: pentru iteme din meniu (un item simplu vs un combo complex)

Observer: pentru notificarea statusului comenzii (client ↔ status livrare)

Strategy: pentru metode de plată (card, cash, online)

Factory Method: pentru crearea comenzilor (comandă simplă vs comandă grup)

Command Pattern: pentru acțiuni de adăugare/configurare item

Pagina 7

Ex1 Composite Pattern

- Fractalul este un caz clasic de structură recursivă (ex: un obiect compus din obiecte de același tip)
- Reprezentarea poate include:
 - Leaf – elemente de bază (forme simple)
 - Composite – obiecte formate din mai multe elemente (inclusiv alte compozite)

Responsabilități:

- Add / Remove / Iterate se află în clasa CompositeFractal
- Se aplică uniformitate de tratament: clientul nu distinge între formă simplă și compozită

```
interface FractalComponent {
```

```
    void render();
```

```
}
```

```
class SimpleFractal implements FractalComponent {
```

```
    void render() { /*draw */ }
```

```
}
```

```
class CompositeFractal implements FractalComponent {
```

```
    List<FractalComponent> children;
```

```
    void render() {
```

```
        for (FractalComponent f : children) f.render();
```

```
    }
```

```
    void add(FractalComponent f) {...}
```

```
    void remove(FractalComponent f) {...}
```

```
}
```


Ex2 Pachetele sunt: Stocks Pricer, Stocks Pricer UI, Portfolio UI, Portfolio Application, Pricing, Scenario Manager, Stocks Database, GUI Library.

Principii de design pachete:

- High Cohesion: pare respectat, UI logic separat de logică aplicație
- Low Coupling: pot fi îmbunătățiri – Portfolio Application pare legat de multe componente
- Stable Dependencies Principle: GUI Library e stabilă (nu depinde de altele)
- Acyclic Dependencies Principle: pare OK, nu există bucle

Idei:

- Portfolio Application poate fi împărțit în sub-pachete (pentru pricing/scenario logic)
- Interfațele abstracte între pachete ajută la decuplare

Ex 3 Abstract Factory – extensibility analysis

Ce se extinde ușor:

- Familii de obiecte (ex: UI toolkit – Buttons, Scrollbars): pot crea o familie nouă ușor
- Noi implementări ale familiei: se face adăugând o nouă clasă de tip ConcreteFactory

Ce se extinde greu:

- Adăugarea de noi produse (tipuri) în familia existentă implică modificări în toate ConcreteFactory
- Încalcă Open/Closed Principle la adăugare de produse noi

Ex4 Sistem pentru Remote Pair Programming

Cerințe:

- 2 utilizatori (driver + observer) editează codul în timp real
- Observer vede și interacționează (dar nu editează)
- Posibilitatea de schimbare a rolurilor
- Sincronizare live și salvare cod

Problemă	Soluție
Sincronizare în timp real	Observer Pattern + WebSocket
Schimbare roluri	Strategy Pattern (rol ca strategie)
Stocare și versionare cod	Memento Pattern sau Git-style snapshot

Problemă

Soluție

Acces simultan și unică versiune Singleton + Concurrency Control

Sincronizare:

- Codul editat este salvat periodic (sau live) în memorie și în baza de date
 - Fiecare schimbare este notificată prin Observer
 - Role switching poate însemna schimb de socket ID-uri
-

Pagina 8

Ex1

Pattern recomandat: Event-Driven Architecture (EDA)

Descriere funcțională:

- Senzorii trimit evenimente (mișcare, temperatură etc.)
- Un „event bus” trimite evenimentele către handleri
- Sistemul reacționează: trimite SMS, pornește sprinklere, etc.

Avantaje:

1. Scalabilitate & decuplare: adăugarea de noi acțiuni nu afectează senzorii
2. Asincron & Reacție rapidă la evenimente

Dezavantaje:

1. Debugging complicat – evenimentele asincrone sunt greu de urmărit
2. Ordonarea evenimentelor poate fi imprevizibilă

Ex2

Liskov Substitution Principle (LSP)

Obiecte derivate trebuie să poată fi folosite oriunde se acceptă obiecte de bază fără efecte secundare neașteptate.

Close() pe ProjectTask aruncă o excepție pe un caz unde Task funcționează.

Soluție:

Separăm validarea de implementarea metodei Close().

Variante posibile:

1. Introduce o metodă CanClose()

```

public class Task {

    public Status Status { get; set; }

    public virtual bool CanClose() => true;

    public void Close() {

        if (!CanClose())

            throw new InvalidOperationException("Can't close");

        Status = Status.Closed;  }}

public class ProjectTask : Task {

    public override bool CanClose() => Status != Status.Started;}

```

Ex3 Diagrama arata:

- OldGraphicalLibraryClass și NewGraphicalLibraryClass oferă metode diferite
- NewClass mediază și le unifică

Pattern sugerat: Adapter Pattern

- Permite ca aplicația existentă să funcționeze cu librăria nouă fără modificări mari
- NewClass acționează ca un adapter

dacă NewClass traduce specialFunctionCall() în functionCall() și invers, codul vechi poate fi reutilizat fără schimbări.

Ex4

Design pattern similar cu exception handling chaining

Răspuns: Chain of Responsibility

Permite trimiterea unei cereri printr-un lanț de handleri.

- Un handler poate trata cererea sau o poate pasa mai departe.
- Similar cu try-catch unde un bloc poate prinde sau relansa excepția.

```

abstract class Handler {

    protected Handler next;

    public void setNext(Handler h) { next = h; }

    public abstract void handle(Request req);}

```

Pagina 9

Figlo este o platformă online care:

- Salvează date financiare sub formă de financial cards
- Oferă predicții financiare bazate pe acele date
- Suportă 3 roluri: Client, Expert, Administrator

Ex1

Actori:

- Client
- Expert financiar
- Administrator

Use Cases (exemple relevante):

- Client: register, manage financial cards, generate prediction, export data, request deletion
- Expert: CRUD financial cards, support clients
- Admin: CRUD clients, CRUD experts

Ex2 diagrama arhitecturala

Ex3 secventa

Ex4

Composite Pattern – pentru FinancialCard

Observer Pattern – pentru actualizări de predicție

Pagina 10

Ex1 Încalcă Open/Closed Principle – codul trebuie modificat pentru fiecare nou tip de hashing.

Strategy Pattern

Ex2 Volatilități (zone cu schimbări frecvente):

- Senzori: tipuri, producători
- Reacții: acțiuni automate (frânare, ștergător etc.)
- Decizii AI: învățare comportament
- UI / Dashboard

Arhitectură propusă (SOA + Layered):

[Input Layer]

- SensorService (Speed, Rain, Camera)

[Processing Layer]

- DecisionEngine

- BehaviorAnalysis

[Action Layer]

- BrakeController

- WiperController

- AlertNotifier

[Interface Layer]

- Dashboard

- Mobile App

Ex3

Ex4 Composite Pattern

Pagina 11

Ex1

Cerinte:

- Funcțională: Detectarea anomaliilor comportamentale și trimiterea de alerte în timp real.
- Ne-funcțională: Timp de răspuns scăzut (low-latency) și disponibilitate ridicată.

Arhitectură propusă: Event-Driven + Layered

Event-Driven Architecture (EDA)

- Permite reacții rapide și decuplare între detecție și acțiune.

Ex2

Design Pattern-uri folosite:

- Data Mapper Pattern – PatientMapper mediază între obiecte (Patient, Address) și PatientRecord din DB.
- Association Pattern – Patient ↔ Address (1:1), Diagnostic ↔ Symptom (posibil N:1)

Ex3

Componentă cu criptare flexibilă la salvare

a) Pattern recomandat:

- Strategy Pattern – pentru criptarea variabilă
 - Decorator Pattern – dacă vrei să adaugi criptarea fără a modifica componenta
-

Pagina 12

Ex1

Probleme identificate:

- Pachetele sunt puternic cuplate între ele, cu dependențe circulare sau transversale → încalcă Acyclic Dependencies Principle.
- Comunicarea directă între componente UI, Security, Obstacle Detection → violare a Stable Dependencies Principle.

Corectare:

- Separare clară pe layere: FE, BE, DB, si layered etc
- Introducerea de interfețe pentru decuplare
- Mediator între modulele critice pentru evitare dependențe directe

Ex2

Arhitectură pentru elderly health monitoring system

Volatilități identificate:

- Senzori noi → extensibilitate input
- Algoritmi de detecție comportamentală → se pot schimba
- Canale de notificare (SMS, sunet, lumini etc.) → dependente de echipamente

Arhitectură propusă (SOA + Layered):

Ex3

Data Mapper Pattern – PatientMapper

DTO Pattern – PatientRecord

Association Pattern – între Patient și Diagnostic, Symptom, Address

Ex4

Stil arhitectural recomandat:

Pipe & Filter Architecture

Justificare:

- Separă fiecare pas în componente modulare

- Permite ordonare și reordonare ușoară
- Simplu de integrat noi algoritmi (plug-and-play)

Fiecare modul este un Filter, iar datele trec printr-un pipeline configurabil

Pagina 13

Ex1

- Încalcă Liskov Substitution Principle: clasa de bază oferă o metodă care nu e suportată de majoritatea derivate.
- Încalcă și Single Responsibility Principle: Vehicle face prea multe (și condus, și AI?).

```
interface Vehicle {
    void brake();}

interface SelfDriving {
    Object recognizeRoadSign();}

class ManualCar implements Vehicle {
    int velocity;

    public void brake() { velocity--; }}

class SelfDrivenCar implements Vehicle, SelfDriving {
    int velocity;

    public void brake() { velocity--; }

    public Object recognizeRoadSign() {
        // AI logic
    }
}
```

Ex2

Pattern folosit: Observer Pattern

- Când compania postează un job, toți Seeker-ii suntificați.

Ex3 – la fel, architectural etc

Pagina 14

Ex1

Actori:

- Client – vizualizează/restaurante, plasează comenzi
- Angajat – editează meniul, marchează comenzi ca finalizate
- Sistem – notifică, calculează preț, generează rapoarte

Ex2

Design Patterns:

- Factory – generare rapoarte
- Command – calcule pentru comenzi

Ex3 – diagrama secvente

Ex4

Ex 5 factory dp

Pagina 15

Ex1

Actori principali:

- Guest – doar vizualizează evenimente și oferte
- Normal User – în plus: înregistrare la evenimente, postare și achiziție sculpturi, export rapoarte
- Event Planner – în plus: creează și modifică evenimente

Cazuri de utilizare (exemple):

- vizualizează evenimente (toți)
- înregistrează-te la eveniment (user, planner)
- postează sculptură / cumpără sculptură
- creează eveniment, modifică eveniment (doar planner)
- exportă raport CSV/PDF (user/planner)

Ex2 diagrama clasica cu observer si strategy

- Observer: notificare automată la modificări de evenimente
 - Strategy: formate export (CSV / PDF)
-

Pagina 16

Ex1

Actori:

- Guest: poate doar vizualiza evenimente și oferte
- NormalUser: + se poate înregistra la evenimente, poate cumpăra/posta sculpturi
- EventPlanner: + creează și modifică evenimente, notifică userii

Use Cases:

- Vizualizează evenimente / oferte
- Înregistrează-te la eveniment
- Postează / cumpără sculptură
- Creează / modifică eveniment
- Exportă raport CSV / PDF

Ex2 architectura

Design Patterns:

- Observer – notifică utilizatori la modificări în eveniment
- Strategy – pentru export